

Purple Team Outline

Week 07: Web App Foundations & SQL Injection Mastery

Objective: To bridge the gap between network and application layers by mastering SQL Injection (SQLi).

Task : Web Application Foundations & SQL Injection Mastery

- **Description:** Bridge the gap between **network-level security** and **web application security** by learning how attackers exploit **SQL Injection (SQLi)** vulnerabilities using manual techniques and automation tools.
- **Technical Requirements:**
 - DConfigure a vulnerable web application environment using DVWA or OWASP Juice Shop to simulate real-world web security vulnerabilities.
 - Set up and configure Burp Suite as a proxy to intercept and analyze HTTP/HTTPS traffic between the browser and the web application.
 - Capture, inspect, and modify HTTP requests using Burp Suite components:
 - Interceptor
 - Repeater
 - Intruder
 - Error-Based SQL Injection
 - Union-Based SQL Injection
 - Boolean-Based (Blind) SQL Injection
 - Time-Based SQL Injection
 - Use the automated SQL injection exploitation tool sqlmap to accelerate database enumeration and data extraction.
- **Learning Goal:** Recognize how automation reduces human error and increases the speed of response/attack & Gain the ability to "build your own tools" instead of relying solely on pre-built software.

- **What to Document :**

- **Day 1:**

- Install and configure a vulnerable web application:
 - DVWA **or**
 - OWASP Juice Shop
 - Set up the environment using:
- XAMPP / Docker / Kali Linux
 - Configure database (MySQL/MariaDB) required by DVWA.
 - Set **DVWA Security Level** → **Low**.
 - Verify login and basic functionality.
- **Deliverable**
 - Screenshot showing the vulnerable application running locally.

- **Day 2:**

- Install and launch Burp Suite.
- Configure browser proxy:
 - Proxy: 127.0.0.1
 - Port: 8080
- Enable **Burp Interceptor**.
- Intercept login requests from DVWA or Juice Shop.
- Analyze captured HTTP requests:
 - Headers
 - Cookies
 - Parameters
 - POST data
- **Deliverable**

- Screenshot of intercepted login request.

- **Day 3:**

- Send captured login request to **Repeater**.
- Modify parameters such as:
 - username
 - Password
 - Observe response changes.
- **Intruder**
 - Configure **payload positions** in login parameters.
 - Perform a simple brute force test with wordlists.
- **Deliverable**
 - Screenshot showing a modified request and server response.

- **Day 4:**

- Practice SQLi in DVWA **SQL Injection module**.
- **Error-Based SQLi**, Example payload: ' OR 1=1 --
- Observe database errors revealing structure.
- **Union-Based SQLi**, Find column count: ' ORDER BY 3 --
- Extract data: ' UNION SELECT null, database() –
- **Deliverable:**
 - **Extract**
 - database name
 - table names
 - user records

- **Day 5:**

- Exploit SQLi when errors are hidden.
- **Boolean-Based SQLi**, Example:
 - ' AND 1=1 --
 - ' AND 1=2 –
 - Observe page differences.

- **Time-Based SQLi**, Example:
 - ' AND SLEEP(5) –
 - Observe delay response.
 - **Deliverable:**
 - Demonstrate successful Boolean or time-based injection.
- **Day 6:**
- Automate exploitation and large-scale data extraction.
 - Use sqlmap with captured request from Burp Suite.
 - Example workflow
 - Capture vulnerable request in Burp.
 - Save request as file.
 - Run sqlmap:
 - sqlmap -r request.txt --dbs
 - Further extraction:
 - sqlmap -r request.txt -D dvwa --tables
 - sqlmap -r request.txt -D dvwa -T users --dump
 - **Deliverable:**
 - Dumped database table (users/passwords)